SOFTWARE ENGINEERING
& PROJECT MANAGEMENT

DAY 12

Requirements Analysis, Prioritisation & Conflict Resolution

Student Book • Module II: Requirements Engineering

Learning focus
Transform raw stakeholder feedback into clear, consistent, feasible, prioritised, and verifiable requirements that can be documented in an SRS.

Running case study: College Event Management System (CEMS)

Designed for classroom projection, discussion, practical application, and revision.

1. Learning Outcomes

By the end of this session, you should be able to:

• Explain the purpose of requirements analysis and how it differs from elicitation.

• Identify ambiguity, incompleteness, inconsistency, duplication, and infeasibility in requirements.

• Use prioritisation techniques such as MoSCoW and value-versus-effort analysis.

• Resolve stakeholder conflicts through clarification, trade-offs, and documented decisions.

• Produce a refined requirement set and record assumptions, constraints, dependencies, and open issues.

Core idea: elicitation discovers candidate needs; analysis examines them carefully so the team can decide what they mean, whether they fit together, how important they are, and whether they can realistically be delivered.

2. What Is Requirements Analysis?

Requirements analysis is the systematic examination and refinement of requirements gathered from stakeholders and other sources. It helps the team understand the problem, discover missing information, remove contradictions, model important behaviour, assess feasibility, and prioritise what should be delivered.

Elicitation vs analysis

Elicitation	Analysis
Discovers what stakeholders say, do, need, and expect.	Checks whether candidate requirements are clear, complete, consistent, feasible, and valuable.
Example: a coordinator says registrations should close when the event is full.	Analysis asks what counts as full, how simultaneous registrations are handled, and what message the student sees.
Output: interview notes, observations, candidate requirements.	Output: refined requirements, priorities, models, resolved conflicts, and open questions.

Analysis is not simply rewriting sentences. It involves reasoning about business rules, data, workflows, constraints, exceptions, and interactions between requirements.

3. Quality Checks for Requirements

Quality	Question to ask	Example problem
Correct / necessary	Does this represent a real stakeholder need or constraint?	A feature is requested without a clear purpose.
Complete	Is enough information present to understand the expected behaviour?	“Allow cancellation” has no deadline or rule.
Unambiguous	Would different readers interpret it in the same way?	“Show recent events” does not define recent.
Consistent	Does it conflict with another requirement or rule?	One requirement permits unlimited registrations; another enforces a strict capacity.
Feasible	Can it be achieved within technology, budget, time, and policy constraints?	A requirement promises unlimited scale with a very small budget.
Verifiable	Can the team inspect or test whether it is satisfied?	“The system must be excellent.”
Traceable	Can the requirement be linked to its source and related work?	A requirement has no recorded owner or rationale.
Prioritised	Is its relative importance known?	A minor visual enhancement is treated as equal to access control.

4. Find Ambiguity and Missing Details

Ambiguity occurs when a statement can be interpreted in more than one reasonable way. Incomplete requirements omit conditions, rules, or outcomes needed to build and test the system.

Raw statement	Questions to resolve	Improved direction
The system shall show recent events.	How recent? Which time zone? Are past events included? Is the list sorted?	Define a time window and ordering rule, for example upcoming published events sorted by start date.
Students can cancel registrations.	Until when? Are cancellations recorded? Does capacity reopen? Is the coordinator notified?	Define the deadline, status change, capacity behaviour, and notification rule.
The system should be fast.	Which action? Under what load? What response-time target?	Define measurable response time for a named operation and test conditions.
The system shall support large events.	How many attendees? What counts as support: registrations, check-in, reporting?	Define expected event size and required operations under that load.

Useful follow-up prompts

• What exactly does this term mean in this context?

• What should happen before, during, and after this action?

• What if the normal condition is not met?

• Who is allowed to perform this action?

• What evidence would prove that the requirement is satisfied?

• Which rule or stakeholder source supports this requirement?

5. Detect Inconsistency, Duplication, and Dependencies

Inconsistency

Two requirements are inconsistent when they cannot both be satisfied under the same conditions. Example: one statement says all students can register for any event, while another says only students from a specific department can register. The team must clarify whether the rules apply to different event types or whether one statement is incorrect.

Duplication

Duplicate requirements express the same need more than once, sometimes using different wording. Duplicates can cause confusion when one copy changes and the other does not. Keep one authoritative requirement and link relevant sources where needed.

Dependencies

A requirement may depend on another requirement, service, data source, approval, or technical capability. For example, sending a confirmation email depends on having a valid email address and an email delivery service.

Issue	CEMS example	Analysis action
Conflict	Capacity must never be exceeded, but two users may register for the last seat at the same time.	Define an atomic capacity check and expected outcome for concurrent requests.
Duplicate	Two requirement IDs both say students can view published events.	Consolidate or clarify distinct meanings and preserve traceability.
Dependency	Registration confirmation depends on successful registration and a notification service.	Document dependency and define behaviour if notification delivery fails.

6. Model Requirements to Understand Behaviour

Models help stakeholders reason about complex processes without reading only long paragraphs. A model should clarify the problem, not hide unresolved questions.

Example: event registration flow

Step	Expected behaviour / question
1. Browse	Student views published events and their eligibility, date, venue, and capacity status.
2. Select	Student selects an event and requests registration.

Step	Expected behaviour / question
3. Validate	System checks authentication, eligibility, duplicate registration, and remaining capacity.
4. Decide	If valid and capacity remains, save registration; otherwise reject with an appropriate reason.
5. Confirm	Show the result. If notifications are in scope, send confirmation after the registration outcome is known.

This flow reveals questions that need decisions: Can students register without signing in? What happens if capacity changes during submission? Does a notification failure cancel the registration? Record these questions instead of guessing.

7. Prioritisation: MoSCoW

Prioritisation helps the team decide what is essential for a release and what can wait. MoSCoW groups requirements into four categories.

Category	Meaning	CEMS example
Must have	Essential for the release to be viable or compliant.	Prevent over-capacity registration; enforce authorised publishing.
Should have	Important, but the release can work temporarily without it.	Send an email confirmation if an alternative confirmation is available.
Could have	Useful enhancement if capacity remains.	Show suggested events based on student interests.
Won't have this time	Explicitly excluded from the current release.	Integrate with an external ticket marketplace, if outside agreed scope.

“Won't have this time” does not necessarily mean the feature will never be built. It means the team has made a scope decision for the current release.

8. Other Prioritisation Approaches

Value versus effort

Compare the expected benefit of a requirement with the effort, cost, risk, and dependencies involved. A simple matrix can support discussion, but the estimates should be based on evidence and team judgement.

	Lower effort	Higher effort
Higher value	Strong early candidate	Plan carefully; may be strategic
Lower value	Consider if capacity remains	Usually defer or reconsider

Risk and dependency considerations

• A legally or security-mandated requirement may be essential even if users rarely notice it.

• A low-effort feature may still be risky if it depends on an unstable external service.

• A requirement that unlocks several others may need to be delivered early.

• Stakeholder preference is important, but it should be balanced with risk, feasibility, and business goals.

9. Resolving Stakeholder Conflicts

Stakeholder conflicts are normal because groups have different responsibilities and success measures. The analyst should make the conflict explicit, understand the underlying goals, and help the authorised decision-makers choose a workable rule.

Example conflict: event capacity

Students want registration to remain open. Coordinators need a firm capacity limit for venue safety. Rather than choosing one group’s preference without analysis, clarify the underlying goals and agree on a rule such as: registration remains open until the published capacity is reached; then the system closes registration and displays a full-capacity message.

A practical resolution process

• State the conflicting requirements neutrally and identify their sources.

• Ask each stakeholder what goal or risk the requirement is intended to address.

• Identify constraints such as safety, law, budget, schedule, and technical limits.

• Generate options and evaluate their benefits, costs, and risks.

• Agree on a decision-maker or escalation route when consensus is not possible.

• Record the decision, rationale, affected requirements, and any deferred issue.

• Update the requirements and validate the revised wording with affected stakeholders.

The analyst facilitates a reasoned decision; they should not silently resolve business-policy disputes by making assumptions.

10. Assumptions, Constraints, and Open Issues

Item	Definition	CEMS example	What to do
Assumption	A belief treated as true until confirmed.	Every student has a college email account.	Confirm with IT or define an alternative.
Constraint	A boundary that limits the solution.	The system must use the college-approved identity provider.	Record the source and assess its effect on design.
Open issue	A question that has not yet been decided.	Can a student cancel after the deadline?	Assign an owner and due date; track to resolution.

Do not hide uncertainty. A visible open issue is safer than an undocumented assumption that later becomes an expensive surprise.

11. Practical Example: Refine a Requirement Set

Raw stakeholder notes:

• “Students should register quickly.”

• “No event can have too many people.”

• “Coordinators can edit events.”

• “Send messages when something changes.”

• “The system should be secure.”

Candidate	Analysis questions / refinement
Registration speed	Which step and target? Define response time or usability criteria with stakeholders.
Capacity	Define capacity per event, concurrent-registration handling, and the full-capacity response.
Event editing	Which coordinator roles? Which fields? Are published-event changes audited or communicated?
Change notifications	Which changes trigger notifications, to whom, through which channel, and what if delivery fails?
Security	Define authentication, role permissions, data access, logging, and applicable policy requirements.

Notice that analysis does not invent every answer. It identifies exactly what must be clarified, then records the agreed rule once the right stakeholders have decided.

12. Classroom Activity

Activity: Requirements analyst challenge

Work in small groups. Analyse the following statements for a college event application:

• “The system should show important events.”

• “Students can register even when the event is full.”

• “Only coordinators can publish events.”

• “The system shall send a notification for every change.”

• “Registration must be easy and fast.”

For each statement:

• Identify ambiguity, conflict, missing detail, or a clear requirement.

• Write at least two clarification questions.

• Propose a refined requirement or record an open issue if a decision is needed.

• Assign a provisional MoSCoW priority and explain your reasoning.

• Identify one acceptance test or verification method.

Expected output: a requirements-analysis table with issue, questions, refined wording, priority, source, and test idea.

13. Knowledge Check

1. How does requirements analysis differ from elicitation?

Elicitation discovers candidate needs; analysis examines and refines them for clarity, consistency, feasibility, completeness, and priority.

2. Give two signs of an ambiguous requirement.

A key term has multiple meanings, or conditions and expected outcomes are not defined.

3. What is a dependency?

A relationship where one requirement or capability relies on another requirement, service, data, or decision.

4. What does MoSCoW stand for?

Must have, Should have, Could have, and Won't have this time.

5. How should an analyst handle a stakeholder conflict?

Clarify goals and constraints, compare options, obtain an authorised decision, and document the rationale.

6. Why record assumptions and open issues?

To make uncertainty visible, assign responsibility, and prevent unconfirmed beliefs from being treated as facts.

7. Is a low-effort requirement always a high priority?

No. Value, risk, dependencies, obligations, and release goals also matter.

8. Why should requirements be verifiable?

So the team can determine through evidence whether the agreed need has been satisfied.

14. Glossary

Term	Meaning
Requirements analysis	Examining and refining candidate requirements for quality and fit.
Ambiguity	Wording that allows more than one reasonable interpretation.
Inconsistency	A conflict between requirements, rules, or constraints.
Dependency	A relationship in which one item relies on another.
MoSCoW	A method for categorising requirements by release priority.
Trade-off	A decision that balances competing goals, costs, risks, or constraints.
Assumption	A belief treated as true but not yet confirmed.
Constraint	A boundary or restriction the solution must respect.
Open issue	An unresolved question requiring a decision or further evidence.

15. Homework and Deliverables

Use CEMS or another approved software system.

• Collect eight raw stakeholder statements or use the sample statements from this chapter.

• Analyse each for clarity, completeness, consistency, feasibility, and verifiability.

• Rewrite at least five statements as refined requirements or record them as open issues.

• Prioritise the refined requirements using MoSCoW and justify each priority.

• Document one stakeholder conflict, at least two options, the recommended decision, and its rationale.

• List three assumptions, constraints, or dependencies and explain how each will be checked.

• Write acceptance-test ideas for at least four refined requirements.

Next session: Software Requirements Specification (SRS)—structure, writing standards, use cases, and traceability.